Criterion A: Planning

1. The Client's Problem

My client, [REDACTED FOR PRIVACY], is an internet content creator who's main profession is creating live streams and videos on the internet. Whenever [REDACTED FOR PRIVACY] would like to start a stream, he uses third-party services to broadcast his livestream to his audience. During the livestream he is also able to communicate with his viewers using the integrated live chat. If a viewer misbehaves by being offensive or sending inappropriate messages in the chat, my client may choose to block that person from writing for a certain amount of time or even permanently.

Although this solution has worked for [REDACTED FOR PRIVACY]. for a long time, he is now noticing the flaws in the current it which are making him reconsider if it really fits him. The main reasons are that he'd like to have extended management over the website. Furthermore, my client is forced to use domains of the third-party services, even though he'd prefer to use his own to make him look unique. Lastly, my client would like to have more customization options other than the title and descriptions which would allow him to make his website look unique.

Because of these limitations, [REDACTED FOR PRIVACY] has requested me to create a solution that solves the above mentioned problems and contains other features such as custom CSS in order to customize his live stream page.

[228 words]

2. The Rationale of The Solution

As a solution for [REDACTED FOR PRIVACY]. I have proposed a custom live streaming program which will act as a server being able to receive the live stream and will also act as a web server in order to broadcast the live stream to his audience and handle the live chat.

For the backend I'll be using Rust programming language as it's a compiled language, it's extremely fast which is important for handling a lot of data such as a live stream and it's viewers. Additionally, it's memory safe making it more secure and prevents system crashes due to memory leaks.

To process the video stream I'll be using the gstreamer library for Rust. Gstreamer is a multimedia framework which makes it simple to compress the livestream and convert it into the required format for other viewers. The robust framework also makes it hard for any security vulnerabilities to enter the video pipeline.

Regarding the front-end I'll use JavaScript as it is the official language used in browsers, it makes it effortless to modify elements and add functionality to the website.

Finally, I'll use SQLite for the database. SQLite is lightweight and fast considering the volume of data that will be handled. SQLite is also slightly more secure than

other databases because SQLite is a single file meaning that it cannot be ‘hacked into’ unlike other databases.

[233 words]

3. Success Criteria

1. Program has the ability to receive live stream from external applications such as OBS (Open Broadcasting Software)
2. Users have the ability to watch the live stream using the website.
3. Users are able to create accounts.
4. Users are able to login into the accounts.
5. Registered users have the ability to send messages in a live chat room.
6. Administrator has the ability to manage the stream title and description
7. Administrator has the ability to modify the CSS file to change the website's look.
8. Program contains metadata inside the HTML code.[Appendix A2 email response]